Базы данных

А8. Структуризация инструкций SQL

Московский государственный технический университет
имени Н.Э. Баумана

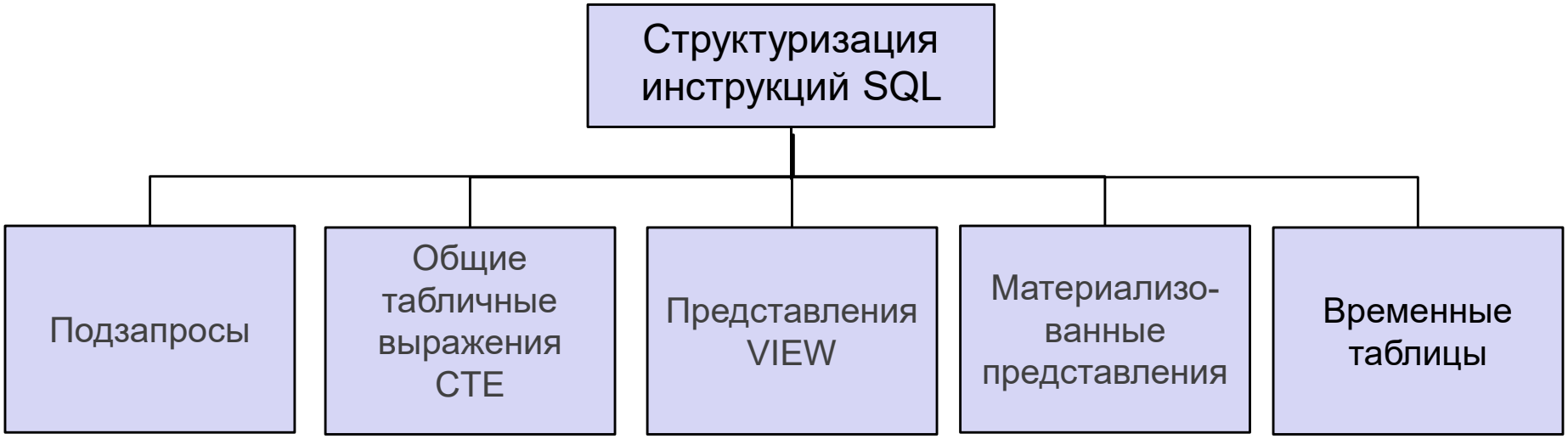
Факультет ИБМ

Фев 2025 года

Москва

Артемьев Валерий Иванович © 2025

Способы структуризации инструкций SQL



Неименованные внутренние запросы, вложенные в основной запрос или другие подзапросы, для получения промежуточных результатов *без сохранения*

Именованные внутренние запросы, вложенные в начале основного запроса, для получения и *хранения промежуточных результатов в рамках основного запроса*

Сохранённые именованные внешние запросы для получения промежуточных результатов *без сохранения*

Сохранённые именованные внешние запросы для получения и *сохранения* промежуточных результатов

Локальные и глобальные таблицы для получения и *сохранения* промежуточных результатов

Подзапросы в языке SQL

Неименованные внутренние запросы, вложенные в основной запрос или другие подзапросы, для получения промежуточных результатов *без сохранения.* Они могут вкладываться внутрь инструкций SELECT, INSERT, UPDATE и DELETE.

Виды подзапросов:

- *скалярный подзапрос* возвращает одно значение
- подзапрос, возвращающий *список значений*
- *строчный подзапрос* возвращает одну строку
- *табличный подзапрос,* возвращающий несколько строк.

Подзапрос заключается в скобки и выполняется раньше родительского запроса. Допускает несколько уровней вложенности.

Применяются для фильтрации и вычисления данных на основе динамических условий. Альтернативные решения через соединения, не всегда более лаконичные.

Понимание подзапросов – это переход от умения писать простые запросы к способности решать реально сложные задачи.

Подзапросы в инструкции SELECT

Подзапрос может появляться в SELECT в качестве:

1) вычисляемого поля в списке вывода SELECT
2) таблицы в выражении FROM
3) значения или списка значений в условии WHERE
4) значения в условии HAVING.

Не допустимо вкладывать подзапросы в ORDER BY.

SELECT *список вывода, (подзапрос)*
FROM *(подзапрос)*
WHERE *условия фильтрации (подзапрос)*
GROUP BY *список группировки*
HAVING *фильтр с агрегатом (подзапрос)*
ORDER BY *список элементов сортировки*

Пример подзапроса в списке вывода инструкции SELECT

Нужно в одном выводе получить сводные данные из разных таблиц.

Подзапросы для получения агрегатов из разных источников

```
 SELECT
(SELECT COUNT(*)
   FROM  Отделы AS кол_отделов),
(SELECT COUNT(*)
    FROM Сотрудники AS численность),
(SELECT SUM(оклад)*12
    FROM Штаты AS ФОТ);
```

1) В подзапросах вычисляются агрегаты *кол_отделов*, *численность*, *ФОТ* из разных таблиц
2) Вычисленные значения подставляются в список вывода

Пример подзапроса для таблицы в FROM инструкции SELECT

Найти максимум средней зарплаты по отделам.

Подзапрос для получения средних зарплат по отделам

```
SELECT MAX(ср_зарплата)
  FROM
(SELECT AVG(оклад) AS ср_зарплата
  FROM Штаты
GROUP BY отдел_ид) AS Зарплаты_отделов;
```

1) В подзапросе вычисляется столбец *ср_зарплата* по отделам
2) В полученной таблице с одним столбцом будем искать максимум

Пример скалярного подзапроса в фильтре WHERE инструкции SELECT

Получить все товары, у которых цена выше средней.

```
SELECT *
  FROM Товары
 WHERE цена >
(SELECT AVG(цена)
  FROM Товары);
```

Подзапрос для получения средней цены товара

1) В подзапросе вычисляется средняя цена товара
2) Вычисленное значение используется при отборе товаров с ценой выше средней цены

Напишите запрос для поиска товаров с минимальной ценой

Пример подзапроса со списком значений в фильтре WHERE инструкции SELECT

Найти товары, которые никогда не заказывали.

```
SELECT название
  FROM Товары
 WHERE товар_ид NOT IN
(SELECT товар_ид
   FROM Заказы);
```

Подзапрос для получения средней цены товара

1) В подзапросе составляется список заказанных товаров
2) Выбираются товары, которые не попали в этот список

Пример запроса с несколькими уровнями подзапросов в WHERE для SELECT

Найти всех сотрудников, работающих в отделах, расположенных в Казани.

```
 SELECT emp_id, first_name, last_name
    FROM Employees
  WHERE dep_id IN
(SELECT dep_id
    FROM Departments
  WHERE location_id IN
(SELECT DISTINCT location_id
    FROM Locations
 WHERE UPPER(city) = ‘КАЗАНЬ’ )
);
```

1-й уровень подзапроса для получения всех отделов на найденных площадках Казани

2-й уровень подзапроса для нахождения всех площадок в городе Казань

1) Сначала находятся все площадки в Казани
2) Затем по этому списку площадок ищутся все отделы
3) Наконец, по списку этих отделов выбираются сотрудники.

Пример подзапроса существования EXISTS в фильтре WHERE в SELECT

Выбрать клиентов, у которых был хотя бы один заказ.

```
SELECT имя_клиента
  FROM Клиенты
 WHERE EXISTS
(SELECT 1
   FROM Заказы
 WHERE Заказы.клиент_ид = Клиенты.клиент_ид);
```

Подзапрос для определения существования записей, удовлетворяющих условию

1) В подзапросе получаем список признаков наличия заказа от клиента
2) Выбираются клиенты, для которых существует признак наличия заказа

Пример коррелированного подзапроса в фильтре WHERE инструкции SELECT

Коррелированный подзапрос зависит от данных родительского запроса и выполняется для каждой строки родительского запроса.

Найти сотрудников, у которых заплата выше средней в их отделе.

```
SELECT E1.last_name, E1.salary, E1.departament
   FROM  Employees AS E1
 WHERE E1.salary >
(SELECT AVG(E2.salary)
   FROM Employees AS E2
WHERE E2.dep_id =E1.dep_id);
```

Подзапрос для получения средней цены товара

Пример скалярного подзапроса в фильтре HAVING инструкции SELECT

Посчитать заказчиков в San Jose с рейтингом выше среднего.

```
SELECT rating, COUNT ( DISTINCT cnum )
  FROM Customers
GROUP BY rating
  HAVING rating >
 (SELECT AVG (rating)
   FROM Customers
  WHERE city = ‘ San Jose’);
```

Подзапрос для получения среднего рейтинга заказчика

Коррелированные и некоррелированные подзапросы

Различие между ними должен понимать каждый аналитик данных.

Некоррелированный подзапрос – *независимый*, выполняется один раз, и его результат передаётся в родительский запрос.

Такие запросы, как правило, более эффективны. Большинство примеров ниже будут некоррелированными.

Коррелированный подзапрос *зависит от данных родительского запроса*, выполняется многократно для каждой строки результата родительского запроса.

Такие запросы могут быть медленными на больших объёмах данных, так как являются аналогом цикла.

Подзапросы в инструкции UPDATE

Инструкция UPDATE используется для обновления записей в таблице.

Подзапрос может применяться:

1) скалярный подзапрос для значения поля в операторе SET
2) *скалярный подзапрос / подзапрос списка значений* в фильтре WHERE для обновления.

UPDATE *имя_таблицы*
SET *поле1* = *(скалярный подзапрос)*,
поле2 = *значение2*, ...
WHERE *условие (скалярный подзапрос / подзапрос списка значений)*;

Пример подзапроса в инструкции UPDATE

Обновить цену товара 234165 в заказе, установив минимальную цену товаров, увеличенную на 10%.

```
UPDATE Заказы
      SET цена =
(SELECT MIN(цена)
   FROM Товары)*1,1
 WHERE товар_ид=234165;
```

Подзапрос для получения минимальной цены товаров

1) В подзапросе определяется минимальная цена товаров
2) Цена товара 234165 в заказе устанавливается равной минимальной цене, увеличенной на 10%.

Подзапросы в инструкции INSERT

Используется для вставки новых записей в ранее определённую таблицу.

Подзапрос может применяться:

1) скалярный подзапрос для столбца в VALUES
2) табличный подзапрос в VALUES

INSERT INTO *имя_таблицы*
(*столбец1*, *столбец2*, ...)
VALUES (*значение1*, *(скаляр. подзапрос)*, ...);

INSERT INTO *имя_таблицы*
VALUES *(табличный подзапрос);*

Пример подзапроса в инструкции INSERT

Данный запрос суммирует все сегодняшние продажи и вносит их в итоговую таблицу.

```
INSERT INTO Sales_summary (sale_date, total_sales)
      VALUES
     (SELECT TODAY, SUM(amount)
        FROM Sales
      WHERE sale_date = TODAY);
```

Подзапрос для получения итога продаж за день

Подзапросы в инструкции DELETE

Используется для обновления записей в таблице.

Подзапрос может применяться:

1) скалярный подзапрос для значения поля в фильтре WHERE
2) подзапрос списка значений в фильтре WHERE для удаления.

DELETE FROM *имя_таблицы*
WHERE *условие (скалярный подзапрос / подзапрос списка значений)*;

Пример подзапроса в инструкции DELETE

Нужно удалить заказы из таблицы NewOrder, для которых advance_amount меньше максимального значения advance_amount из таблицы Orders.

```
DELETE FROM NewOrder
        WHERE advance_amount <
      (SELECT MAX(advance_amount)
         FROM Orders);
```

Преимущества и недостатки подзапросов

Читаемость

Грамотно используемые подзапросы делают код логичным и легко сопровождаемым.

Не злоупотребляйте вложенностью подзапросов в результате запрос может стать кошмаром. Лучше используйте CTE, представления и временные таблицы. Они позволяют именовать подзапрос и вызывать его несколько раз в основном запросе, что улучшает структуру кода.

Инлайн-запросы

Подзапросы позволяют использовать результат напрямую без создания временной структуры.

Гибкость:

Их можно использовать в операторах SELECT, INSERT, UPDATE и DELETE

Производительность

Подзапросы, особенно коррелированные, могут быть медленными.

Часто запрос с JOIN может быть эффективным.

Всегда смотрите план выполнения запроса (EXPLAIN PLAN).

Когда использовать подзапросы?

Когда вычисление простое и не требует повторного использования.
Когда необходимо фильтровать данные на основе результатов другого запроса.

Подзапросы — не столько синтаксическая конструкция для запросов, сколько способ мышления, позволяющий структурировать сложную задачу на последовательные шаги, писать мощные, гибкие и выразительные SQL-запросы.

Практикуйтесь, сравнивайте с альтернативами, быстро почувствуете, в какой ситуации какой инструмент будет наиболее уместным и эффективным.

Общие табличные выражения CTE

Common Table expression – именованные внутренние запросы, вложенные в начале основной инструкции, применяются для получения и хранения промежуточных результатов *в рамках основного запроса.*

Их определения предваряют инструкции SELECT, INSERT, UPDATE и DELETE.

WITH *имя_CTE_1* AS
(подзапрос_1),
имя_CTE_2 AS
(подзапрос_2), …

Основная инструкция *обращается к* CTE по именам как к таблицам;

Пример с использованием общих табличных выражений CTE

Считаем количество уникальных пользователей, которые совершили хотя бы одно действие на сайте/ в приложении за день.
Метрика DAU показывает "живую" аудиторию и если падает, то пользователи уходят.
Смотрим на целый период, например, неделю с изменениями.
Создаём CTE под именем **Daily_users** для подсчёта метрики DAU, а также её в разбивке по мобилам и десктопам для каждого дня за последнюю неделю.
В основном запросе обращаемся к **Daily_users** как таблице и определяем изменения DAU.

Расчёт числа активных пользователей за день DAU с помощью CTE

```
WITH Daily_users AS (
-- Считаем DAU для каждого дня за последнюю неделю и по типам устройств
SELECT event_date,
            COUNT(DISTINCT user_id) AS DAU,
COUNT(DISTINCT CASE WHEN device_type = 'mobile' THEN user_id END)
      AS mobile_DAU,
          COUNT(DISTINCT CASE WHEN device_type = 'desktop' THEN user_id END)
     AS desktop_DAU
FROM User_events
WHERE event_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
GROUP BY event_date)

-- Основной запрос
SELECT event_date, DAU, mobile_DAU, desktop_DAU,
-- Считаем изменение в процентах к прошлому дню с помощью LAG
            ROUND( (DAU / LAG(DAU) OVER (ORDER BY event_date) -1) * 100.0, 2 )
      AS daily_change_percent
FROM Daily_users ORDER BY event_date DESC;
```

Преимущества CTE

Читаемость

CTE упрощают сложные запросы, разбивая их на логически связанные части, что делает код легче воспринимаемым.

Избежание многократного выполнения:

При запросе в нескольких местах, использование CTE позволяет сохранить результаты в рамках основного запроса, что приводит к более эффективному выполнению.

Рекурсия:

CTE поддерживают рекурсивные запросы, что позволяет выполнять сложные иерархические запросы (например, при работе с родительскими и дочерними записями).

Модульность:

CTE помогают инкапсулировать логику, что делает её повторно используемой в рамках одного запроса, а также упрощает поддержку и изменение SQL кода.

Когда использовать CTE

При работе с сложными соединениями или множеством агрегаций, где повторное выполнение одной и той же подзадачи может значительно снизить производительность.

В ситуациях, когда необходимо создать рекурсивный запрос (например, для извлечения данных из иерархической структуры).

Для повышения читаемости и структурированности запросов, особенно если они содержат несколько уровней вложенности.

Представления VIEW в языке SQL

Представления VIEW – сохранённые именованные внешние запросы, результаты их актуализируются при каждом обращении.

Создаются с помощью инструкций CREATE VIEW на основе результатов выполнения запросов SELECT.

CREATE VIEW *название_представления* **AS**
Оператор SELECT;

Пространство для хранения не выделяется, оно существует лишь во время сеанса приложения с СУБД.

В запросах представления используются как таблицы, но при обращении к ним СУБД выполняет сохранённый запрос.

Применяется:

- упрощение сложных запросов
- сокрытия структуры данных
- ограничение доступа к чувствительным данным.

Пример применения представления VIEW

```
CREATE VIEW Daily _users
-- Считаем DAU для каждого дня за последнюю неделю и по типам устройств
SELECT event_date,
            COUNT(DISTINCT user_id) AS DAU,
COUNT(DISTINCT CASE WHEN device_type = 'mobile' THEN user_id END)
       AS mobile_DAU,
           COUNT(DISTINCT CASE WHEN device_type = 'desktop' THEN user_id END)
      AS desktop_DAU
FROM User_events
WHERE event_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
GROUP BY event_date;

-- Основной запрос
SELECT event_date, DAU, mobile_DAU, desktop_DAU,
-- Считаем изменение в процентах к прошлому дню с помощью LAG
            ROUND( (DAU / LAG(DAU) OVER (ORDER BY event_date) -1) * 100.0, 2 )
       AS daily_change_percent
FROM Daily_users ORDER BY event_date DESC;
```

Преимущества и применение представлений VIEW

Безопасность данных

Ограничивают доступ к конкретным строкам или столбцам.

Повторное использование

Легко повторно использовать в нескольких запросах.

Когда использовать представления

Когда необходимо инкапсулировать сложные запросы.

Для повышения безопасности путём ограничения доступа к данным.

Для избегания дублирования кода

Применение способов структуризации инструкций SQL

Способ структуризации	Применение
CTE	Сложные запросы, рекурсивные запросы, улучшение читаемости, когда нужно повторно использовать результат
Подзапрос	Инлайн-вычисления, фильтрация на основе результатов других запросов.
Представление	Инкапсуляция сложных запросов, повышение безопасности.
Материализованное представление	Улучшение производительности для не часто изменяющихся данных.
Временная таблица	Хранение промежуточных результатов для преобразований.

Терпения и удачи всем, кто связан с базами данных

Спасибо за внимание!

Валерий Иванович Артемьев

МГТУ имени Н.Э. Баумана, кафедра ИУ-5

Банк России
Департамент данных, проектов и процессов

Тел.: +7(495) 753-96-25
e-mail: viart@bmstu.ru